Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University, Chennai)

UCS3513 System Design Laboratory

MICRO PROJECT REPORT

EfficientStock

Scalable Inventory Management System

B.E. Computer Science and Engineering

III Year / V Semester | Academic Year 2026-2027

Submitted by

S.No.	Name	Register Number
1	Adithya S	3122245001007
2	Adithya Sivakumar	3122245001009
3	Amudhan A	3122245001016
4	Amrit Gopinath	312224501017
5	Nithin Karthikeyan	3122245001705

Department of Computer Science and Engineering

October 2026

Abstract

AetherStock is a working inventory-management prototype developed for UCS3513 System Design Laboratory. It integrates product registration, warehouse-wise stock tracking, purchase receipts, sales reservations, inter-warehouse transfers, low-stock alerts and supplier-product discovery. A React interface communicates with a Spring Boot backend, MongoDB persistence and a Redis product-metadata cache.

The design incorporates eight system-design components: a token-bucket rate limiter, logical database sharding, consistent hashing, a key-value cache, Snowflake identifiers, a URL shortener, a supplier web crawler and Trie-based autocomplete. Inventory is routed by a composite product/warehouse key to three active logical databases. A fourth configured shard supports controlled membership changes. Stock availability remains authoritative in MongoDB; transactions jointly commit stock changes, movement records, alerts and business-document state.

Evaluation uses the repository's recorded experiments from 24 October 2026 at 5, 25 and 100 concurrent users. Cache comparisons use three 30-second steady-state repetitions; component HTTP comparisons use short 10-second windows with one repetition. Additional direct benchmarks examine identifier generation, hash distribution and autocomplete. Correctness evidence records 85 passing backend tests.

The results demonstrate workload-dependent trade-offs. Warm Redis eliminates measured product-detail database loads, but improves HTTP performance only at the medium workload in this experiment. Logical sharding adds routing and fan-out overhead on the shared host. A three-to-four-to-three shard change preserves all 803 inventory documents, moving 26.53% in each direction. Hot-stock bursts maintain stock safety while experiencing substantial retryable write conflicts. Redis failure preserves correct product responses through database fallback, with a marked latency penalty.

AetherStock therefore provides an integrated academic demonstration of component selection, transaction safety and measurable scaling trade-offs. Its current boundary is a single backend and one MongoDB deployment: inventory is logically partitioned, while orders and audit data remain centralized. Physical scale-out, coordinated multi-backend operation and several component-specific verification gaps remain future work.

Keywords: inventory management; MongoDB transactions; consistent hashing; logical sharding; Redis; token bucket; Snowflake; supplier crawler; Trie; performance evaluation.

Report basis

Assignment: Topic 4, *Mini Project Problem Statements.pdf*, pp. 8-9 [1]. Implementation and recorded evidence are distinguished throughout. Numerical results reproduce saved observations; explanatory stock examples and architectural diagrams are illustrative.

Contents

Section	Page
Abstract	i
1 Introduction and project scope	1
2 Requirements and component mapping	2
3 System architecture	3
3.2 Data model and consistency	4
4 Algorithms: rate limiting and caching	5
4.3 Sharding and consistent hashing	6
4.4 Unique identifiers and short links	7
4.5 Supplier crawler and autocomplete	8
5 Prototype workflows	9
5.5 Sales and transfer execution	10
6 Evaluation methodology	11
6.1 Product-cache comparison	12
6.2 Resources and Redis failure	13
6.3 Rate-limiter evaluation	14
6.4 Logical-shard read evaluation	15
6.5 Stock correctness and contention	16
6.6 Hash distribution and migration	17
6.7 Snowflake and Trie benchmarks	18
6.8 Verification and evidence coverage	19
7 Scalability, reliability and trade-offs	20
8 End-to-end demonstration	21
9 Conclusion and references	22

List of figures

1. Implemented architecture 3
2. Consistent-hash ring 6
3. Sales and transfer lifecycles.....10
4. Product-cache comparison..... 12
5. Token-bucket admission..... 14
6. Logical-shard read comparison.....15
7. Hash distribution and remapping.....17

1 Introduction and project scope

1.1 Problem statement

An inventory system must answer where products are stored, how many units are available and how each purchase, sale or transfer changes that stock. As warehouses and concurrent requests increase, a simple read-then-write implementation can oversell products, repeat deliveries or lose the link between stock and its audit trail. A system-design solution must address both operational usability and the correctness of concurrent state changes.

The laboratory problem requests an integrated prototype that supports registration, inventory search, warehouse management, orders, transfers and low-stock notifications. It also requires justified component selection, at least three workload levels, baseline comparisons where relevant, resource measurements and a scalability or failure scenario [1].

1.2 Project objectives

Objective	Observable outcome
Reliable stock operations	Reservations and deductions obey stock bounds; failed business transactions roll back.
Partition-aware persistence	Composite keys select inventory owners; shard membership can change with recorded migration results.
Responsive product access	Product metadata caching and prefix search support catalogue browsing and selection.
Controlled request handling	Client buckets admit bounded traffic and expose explicit HTTP 429 rejections.
Evidence-based assessment	Latency, useful throughput, outcomes, resources and correctness are reported together.

1.3 Prototype scope

The implemented system combines an operational React dashboard with component demonstrations. Its business modules include catalogue management, warehouses, purchase and sales orders, transfers, movement history and low-stock alerts. Simulated supplier portals provide reproducible external data. The System Design Lab and Autocomplete Lab expose supporting mechanisms for demonstration.

1.4 Success criteria and boundary

Success means that documented workflows execute through the integrated API, stock and business records reconcile after concurrent activity, and measurements explain the benefit or cost of each tested component. Three logical databases on one MongoDB process demonstrate partition routing; they do not establish additional machine capacity. The assignment's transaction-data partitioning requirement is only partially covered because orders and movement records remain in the core database.

2 Requirements and component mapping

Component choices follow the inventory access pattern and the state each workflow must protect. The mapping below identifies all eight incorporated components and their specific roles.

Table 1. Functional requirements, selected components and justification.

Requirement / challenge	Component	Implementation and justification
Protect APIs during bursts	Rate limiter	Per-client token buckets allow short bursts and bound sustained admission to the backend.
Partition warehouse stock	Sharding	Separate logical MongoDB databases store inventory records; a persistence port keeps business services storage-independent.
Keep placement stable as membership changes	Consistent hashing	SHA-256 ring with virtual nodes routes productId:warehouseId and limits reassignment on shard addition/removal.
Repeated catalogue detail reads	Key-value store	Redis caches product metadata with expiry and invalidation, reducing repeated database reads.
Identify products and business events	Unique ID generator	Snowflake creates prefixed identifiers locally for products, warehouses, orders, movements and alerts.
Share product or related links	URL shortener	Base62 aliases map long URLs to HTTP redirects and record redirect counts.
Discover supplier catalogue information	Web crawler	Strategy-specific parsers collect supplier products, normalize records and submit an import feed.
Search by product name, SKU or category	Autocomplete	A browser Trie performs token-prefix lookup, with a debounced API fallback for missing local matches.

2.1 Cross-cutting requirements

Purchase receipts, sales and transfers also require transactional stock updates, idempotent request handling and movement auditing. A shard router or cache cannot independently provide these properties. Snapshot transactions and conditional stock mutations therefore form the correctness foundation underneath the selected components.

Requirement status: the eight components are present. Inventory partitioning is implemented; partitioning of transaction/order and audit data remains an explicit gap. Component presence does not imply that every possible performance or failure case has been measured.

3 System architecture

AetherStock uses a layered, single-backend architecture. REST controllers validate requests; domain services coordinate workflows; integration ports select cache, identifier and persistence implementations. Figure 1 shows the implemented deployment and data paths.

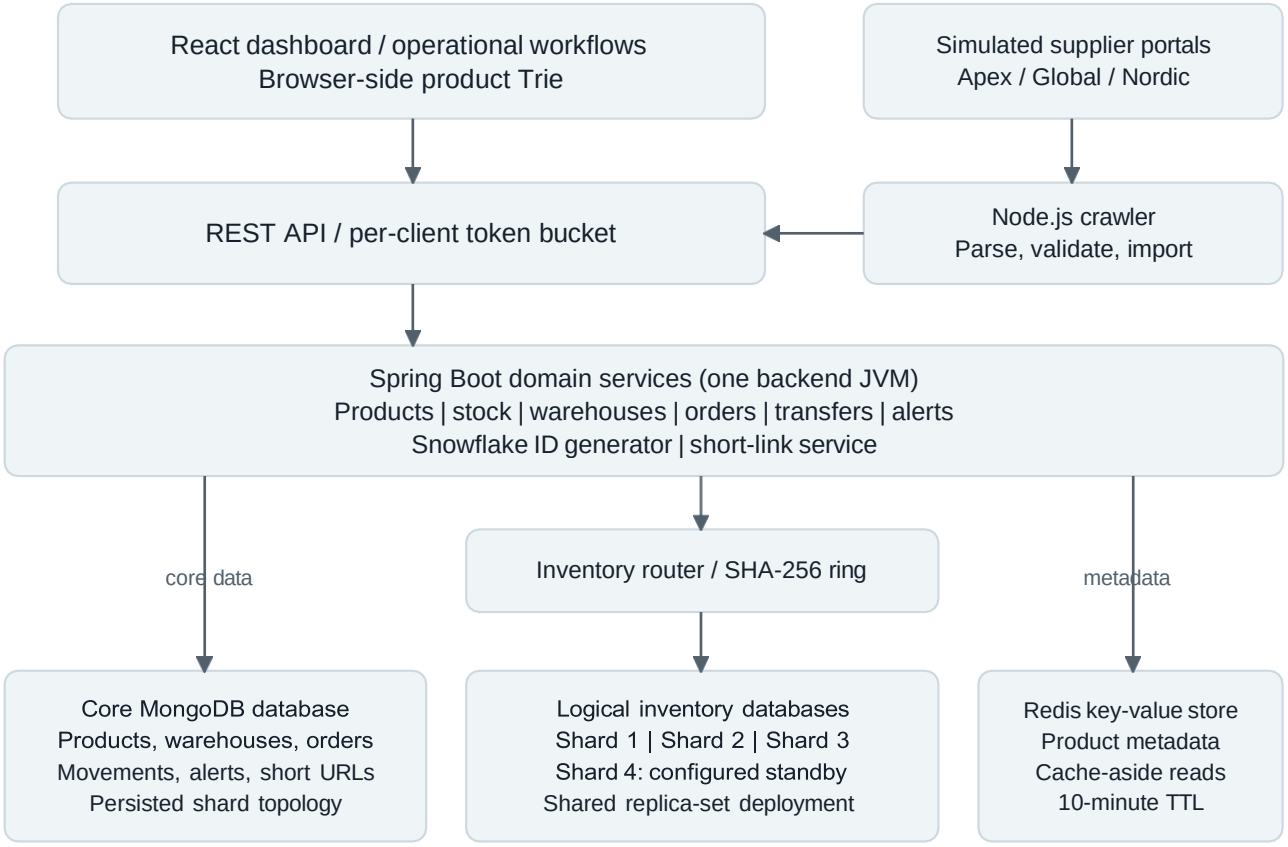


Figure 1. Implemented architecture. All inventory databases share one MongoDB deployment.

3.1 Interaction and transaction boundaries

Product-detail requests use Redis cache-aside access. Inventory availability and mutations use the authoritative persistence router. All logical shard templates share the core MongoDB factory and transaction session, so a business transaction may update inventory and its core order/audit records atomically. The crawler imports catalogue data and may propose a purchase order; goods become physical warehouse stock only on receipt.

Layer	Technology / configuration
Client	React 19 / Vite; catalogue, stock and component demonstration pages
Application	Java 17; Spring Boot 3.3.4; REST API on port 8080 by default
Persistence	MongoDB transaction-capable deployment; Redis metadata cache
External-data simulation	Node.js supplier portal and crawler; portal port 8085 by default

3.2 Data model and consistency

Table 2. Principal records and storage responsibilities.

Record	Important fields / relations	Location
Product / warehouse	Product ID, unique SKU, category, price, reorder level; warehouse ID and active state	Core database
Inventory	Product + warehouse pair, physical quantity, reserved quantity, version, timestamps	Owning logical shard
Purchase order	Supplier, destination, ordered/received items, receipt fingerprints, status	Core database
Sales order / transfer	Item references, quantity, lifecycle status, creation idempotency key	Core database
Movement / alert	Product and warehouse references, movement type and reference ID; threshold and alert state	Core database
Short URL / topology	Unique code, target, clicks and expiry; active/pending shards and ring configuration	Core database

Stock invariants

$$A = P - R \quad | \quad P \geq 0 \quad | \quad 0 \leq R \leq P$$

P: physical units R: reserved units A: available units

A unique product/warehouse index prevents duplicate stock records at an owner. Guarded database updates reserve only available units, release only existing reservations and prevent negative stock. Stock, movements, alert changes, receipt identities and order/transfer state commit or roll back together. Transactions use snapshot reads and majority write concern [2].

Request identity and retries

Creation requests accept stable idempotency keys. An existing key with the same canonical payload returns the previous result; changing the payload produces a conflict. Partial receipts require an explicit identity, and repeated receipt lines are aggregated before remaining quantities are checked. The supplied creation forms retain pending keys in session storage across failed requests and reloads.

Recognized aborted transaction conflicts return HTTP 409. An uncertain commit outcome returns 503 and requires reconciliation or retry of the whole request with the same key. A successful status transition must not survive a failed stock operation. The router holds membership stable until the surrounding inventory transaction completes.

A transaction-capable replica set or compatible MongoDB cluster is required. The local one-member replica set provides transaction semantics without machine-failure redundancy.

4 Algorithms: rate limiting and caching

4.1 Token-bucket admission control

The rate limiter maintains an in-process bucket for each resolved client identity. The default capacity is 10 tokens and the refill rate is 5 tokens per second. A synchronized token operation refills according to elapsed monotonic time, caps the balance at capacity and consumes one token when a request is admitted.

tokens = min(capacity, tokens + elapsed_seconds x refill_rate)
Admit when tokens >= 1; otherwise return HTTP 429.

Rate-limit response headers report the limit and remaining allowance; a rejected request provides retry information. The interceptor protects API traffic. The redirect route is outside the API path, so a short-link click does not traverse that interceptor. A local bucket is inexpensive but applies to one backend process: adding replicas would require shared admission state or an explicitly partitioned quota policy.

4.2 Redis cache-aside product lookup

Step	Product-detail read or update
1. Lookup	Read product metadata from the configured Redis namespace.
2. Miss	Load the product from MongoDB and populate the cache with a 10-minute default TTL.
3. Change	The normal product update path evicts affected metadata.
4. Cache failure	Record the cache failure and fall back to MongoDB; request latency may rise.

Inventory availability bypasses Redis. This avoids making sales decisions from an old cached quantity while concurrent transactions reserve or deduct stock. Legacy stock-cache entries are invalidated after a committed mutation but are not an authority for availability. The product cache is a performance optimization with separate freshness concerns.

Complexity and measurement

Bucket updates require constant work per request; the bucket registry grows with active clients. Cache lookup adds a network exchange and serialization. A cache hit can remove a database read without reducing end-to-end latency when the database working set is already warm. The evaluation therefore measures cache hit ratio, lookup time, database loads, HTTP latency and resource usage instead of assuming that caching is always faster.

4.3 Sharding and consistent hashing

The router hashes the exact composite key **productId:warehouseId** with UTF-8 SHA-256. Each virtual node hashes **shardId#virtualNodeIndex**. The first ring position at or clockwise after the key hash selects the owner, with wraparound at the end of the ring [3].

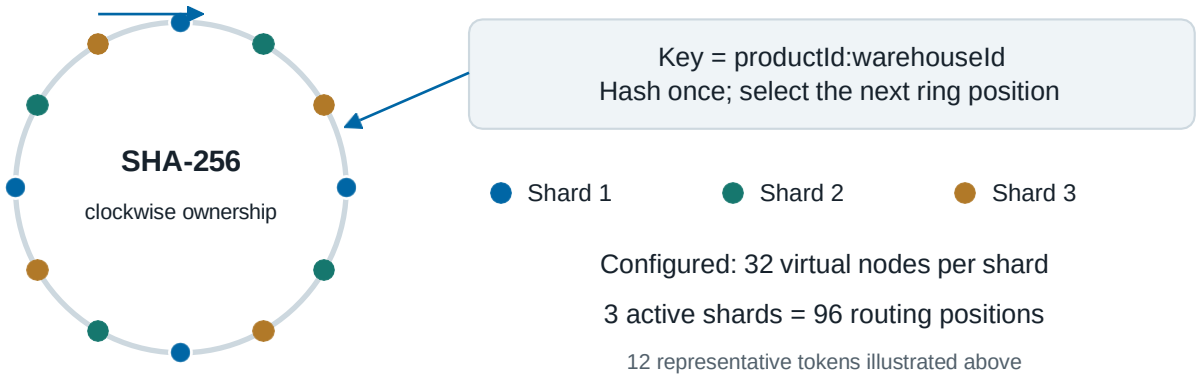


Figure 2. Consistent-hash routing with virtual nodes; schematic token positions.

Lookup and query cost

With S shards and V virtual nodes per shard, a sorted-ring lookup takes O(log(SV)) comparisons. A point read or stock mutation visits the owner. A product-wide or warehouse-wide query visits all active shards and combines results, adding fan-out work. Virtual nodes improve the opportunity for balanced placement but do not create replicas or divide a single hot stock key.

Journaled membership change

Phase	Action and recovery property
Prepare	Acquire the router write lock; persist pending membership and build the target ring.
Copy	Copy records whose owner changes, preserving fields and retaining original copies.
Commit authority	Persist target membership as authoritative while keeping the pending marker.
Clean up	Delete non-owner copies and clear pending state; inventory requests resume.
Resume	Reload persisted topology and repeat the idempotent copy/cleanup stages after interruption.

The local read/write lock spans complete inventory transactions. Migration waits for in-flight work and pauses subsequent inventory operations. A transaction with a pre-change snapshot cannot safely use new routing and receives a retryable conflict. An incomplete migration gates inventory with 503 until recovery. Active membership persists across restarts; ring settings cannot be silently changed for an existing dataset.

4.4 Unique identifiers and short links

Snowflake identifiers

The primary identifier generator combines a millisecond timestamp relative to 1 January 2024, a datacenter identifier, a node identifier and a per-millisecond sequence. Prefixes such as prod_, po_, so_ and mov_ keep identifiers recognizable at API boundaries.

Table 3. Implemented 64-bit Snowflake layout.

Sign	Timestamp	Datacenter	Node	Sequence
1 bit	41 bits	5 bits	5 bits	12 bits
Sign = 0	Milliseconds since epoch	0-31	0-31	0-4095 per ms

```
id = ((timestamp - epoch) << 22) | (datacenter << 17)
   | (node << 12) | sequence
```

Generation is synchronized within the JVM. Sequence rollover waits for the next millisecond. Small backward clock movements are waited out and checked again; larger regressions fail generation. Distinct active generators require distinct datacenter/node identities. Time-sortable numeric values and local generation are useful, but the measured ID rate is not database insertion throughput and does not prove deployment-wide collision freedom.

Base62 short-link mapping

The shortener generates a random six-character code from a 62-character alphabet using SecureRandom. The nominal six-character namespace contains 62^6 = 56,800,235,584 codes. Existing long URLs are looked up before creation. A pre-detected code collision increases the candidate length, with up to five attempts; a unique MongoDB code index guards persistence.

POST /api/shorten creates a mapping. GET /r/{code} resolves it with an atomic click-counter increment and returns HTTP 302. An information endpoint inspects a mapping without adding a click. The controller exposes X-Redirect-Latency-Ms and component counters for evaluation.

Verification boundary

An expiry timestamp is stored with a nominal 90-day default. Lifetime/index behavior, parallel same-URL requests and an insertion-time collision race need dedicated verification. The service checks a code before saving; its retry loop does not catch a duplicate-key race at save time. No saved redirect-load measurements are available in the evaluated evidence; this report therefore does not assign a measured redirect latency to the component.

4.5 Supplier crawler and autocomplete

Supplier discovery pipeline

The Node.js crawler traverses a URL frontier with a visited set. Each queued URL selects a parser strategy: Apex Tech uses product cards with pagination, Global Wholesale uses table rows, and Nordic Supply exposes Product/Offer JSON-LD. The implemented frontier processes fetches sequentially. It normalizes SKU text and numeric fields, validates items and records crawl telemetry.

Pipeline stage	Implemented behavior
Fetch	An 80 ms default client politeness interval, 5 s request timeout and bounded retries.
Retry	Increasing delays of 200, 400 and 600 ms; the current implementation uses linear backoff.
Extract	Supplier-specific card/table parsing or embedded JSON deserialization; discovered pagination is queued.
Normalize	Trim and uppercase SKU, validate required data, normalize categories and non-negative stock.
Integrate	Group feed items by supplier and call POST /api/suppliers/import.

The import service registers newly discovered SKUs, may update the category of an existing product, and can create a purchase order for a chosen warehouse. Supplier-reported stock is procurement information; the import itself does not credit physical stock. Import batches are not one atomic, replay-safe transaction across all catalogue changes and purchase creation.

Trie token-prefix search

The browser indexes tokens from SKU, product name and category, plus a compact SKU form. A prefix lookup walks child maps, then collects product references from the matching subtree, deduplicates by product ID and returns up to eight suggestions. Keyboard navigation and a debounced server query support the search interaction.

> Current search work = prefix traversal + matching-subtree traversal + deduplication.
> Returning eight suggestions does not bound the traversal to eight candidates.

Selective prefixes visit small subtrees; broad prefixes may traverse most of the catalogue. The application initially builds from its fetched product page, so local coverage is bounded by loaded records. The complete-catalogue algorithm benchmark measures the data structure under controlled input and is not a measurement of browser rendering or full application search coverage.

5 Prototype workflows

5.1 Product registration and warehouse setup

A user registers a product with SKU, name, category, price and reorder level, then creates warehouses to represent storage locations. Product and warehouse identifiers come from the Snowflake generator. Registration does not create physical stock. Catalogue search and autocomplete help select existing products while warehouse views show the stock associated with a chosen location.

5.2 Purchase order and receipt

A purchase order identifies a supplier, destination warehouse and positive per-product quantities. Creation validates active warehouse state and duplicate product lines, then stores a CREATED order. A receipt either names delivered quantities with a stable receipt key or receives all remaining items. The service validates all receipt lines before crediting stock and updates the order to PARTIALLY_RECEIVED or RECEIVED.

Table 4. Worked receipt example; illustrative quantities, not benchmark data.

Action	Physical	Reserved	Available	Order result
Initial stock	0	0	0	No receipt
Create PO for 40	0	0	0	CREATED
Receive 15 with key R1	15	0	15	PARTIALLY_RECEIVED
Replay the same R1	15	0	15	Same effect returned
Receive all remaining	40	0	40	RECEIVED

5.3 Adjustment, audit and low-stock alerts

A manual adjustment specifies physical/reserved deltas and a reason. Conditional updates reject a result that would violate stock bounds. Successful mutations append movement records and reevaluate availability against the product reorder level within the transaction. Availability at or below the threshold creates or updates an ACTIVE alert; recovery above the threshold resolves it. A unique partial index limits active alerts to one per pair.

5.4 Operational interface

The dashboard links Product Catalog, Warehouse Inventory, Purchase Orders, Sales Orders, Stock Transfers and Low Stock Alerts. Supplier Crawler, Autocomplete Lab and System Design Lab provide component demonstrations. API failures are surfaced to the user, including rate-limit and stock/conflict responses. Product information and inventory quantities use their respective metadata and authoritative-stock access paths.

5.5 Sales and transfer execution

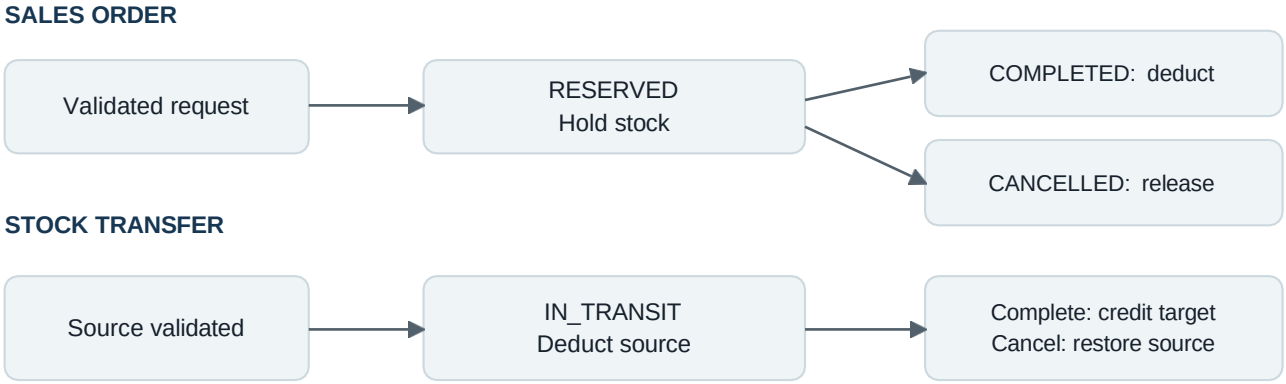


Figure 3. Implemented sales and transfer lifecycles, with stock effects at each transition.

Sales reservation, completion and cancellation

Placing an order validates every product/warehouse line and conditionally reserves its units. Successful creation stores a RESERVED order. Completion deducts both physical and reserved quantities; cancellation releases reservations while leaving physical stock unchanged. The service refuses invalid transitions and must observe successful stock operations before saving a terminal state.

Table 5. Sales example for a 4-unit order; each terminal row is an alternative.

State	Physical P	Reserved R	Available P - R
Before order	10	0	10
RESERVED	10	4	6
Then COMPLETED	6	0	6
Or CANCELLED	10	0	10

Inter-warehouse stock conservation

Initiating a transfer deducts available stock from the source and creates an IN_TRANSIT record. Completion credits the destination; cancellation restores the source. Repeating an already acknowledged terminal action returns its result without applying stock again. Completing an undisbursed transfer is rejected.

For a transfer without external receipts or sales:
source physical + destination physical + in-transit units = constant

In-transit units are represented by the transfer document rather than by warehouse physical stock. A source/destination sum alone temporarily falls during transit and is therefore an incomplete conservation check. Failure-injection and concurrency tests reconcile business records with stock and movements, including cross-logical-shard operations.

6 Evaluation methodology

All numerical experiments below use saved repository evidence dated 24 October 2026 [4-6]. The low, medium and high HTTP workloads are 5, 25 and 100 concurrent users. Separate experiments isolate each mechanism and use an appropriate baseline.

Table 6. Experimental matrix and recorded conditions.

Study	Workload / duration	Comparison or check
Product metadata	1,000 products; 5/25/100 users; 30 s steady window; 3 repeats; 5 s ramp excluded	Redis off vs on; limiter off; same endpoint and product assertions
Inventory reads	100 products x 5 warehouses; 1,000 units/record; 10 s; 1 repeat per user level	Single store vs 3 logical shards; alternating point and product-wide queries
Rate limiting	5/25/100 users; paced 10/50/200 requests/s; 10 s after warm-up	Off vs one shared bucket, capacity 10 and refill 5/s
Stock contention	100 one-unit orders per case; 5/25/100 users; finite bursts	Distinct stock records vs one hot record with 10 units; first attempts only
Membership / failure	25-user shard-change run; separate 5-user Redis-outage run	Add/remove shard and reconcile records; unavailable cache then recovery
Direct algorithms	250,000 IDs/point; 1k/10k/100k keys; 100/1k/10k products	Snowflake vs UUID; ring vs modulo; Trie vs token-prefix linear scan

Environment and metric definitions

Recorded host: Windows 11, 12 logical CPUs, 8 physical cores and approximately 15.65 GiB RAM. The HTTP driver is Apache JMeter 5.6.3 and the backend runs on Microsoft JDK 17.0.17. Benchmark backends use port 8082 and an isolated test MongoDB on 27019; the cache study uses Redis on 6381. Applications, database and load generator share the host.

Successful throughput = admitted successful operations / measured seconds.
Success rate = successful operations / attempts x 100.
Expected 409/429 rejections are reported separately from unexpected errors.

Mean, p95 and throughput describe the specified windows. Cache table p95 values are means of per-run p95s, not a pooled percentile. CPU is normalized to total logical-CPU host capacity; RSS is observed process resident memory. Manifests retain settings, revisions and artifact hashes. Short, same-host runs support local comparisons, not a production-capacity estimate.

6.1 Product-cache comparison

Warm product-detail reads were measured with cache off and on at all three user levels. Every measured response passed status and product-field assertions. Cache-on hit ratio was 100%; measured database loads per request fell from 1 to 0.

Table 7. Cache baseline results: three 30-second repetitions per point [4].

Users	Cache	Mean ms	p95 ms	Success/s	Error %	DB reads / req
5	off	0.935	1.667	4,941.1	0.00	1.0
5	on	1.246	2.667	3,798.5	0.00	0.0
25	off	8.107	15.667	3,057.4	0.00	1.0
25	on	5.644	11.667	4,346.1	0.00	0.0
100	off	15.779	21.000	6,268.6	0.00	1.0
100	on	26.364	32.333	3,757.2	0.00	0.0

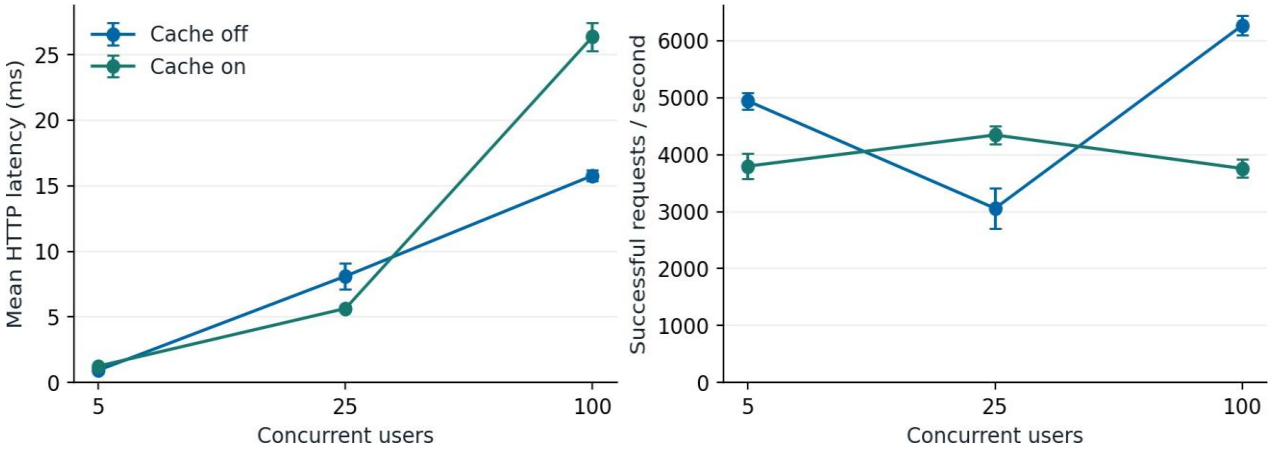


Figure 4. Product-cache latency and throughput; error bars show one between-run standard deviation.

Interpretation

At 25 users, cache-on mean latency fell from 8.107 to 5.644 ms, approximately 30.4%, and successful throughput rose from 3,057.4 to 4,346.1 requests/s, approximately 42.1%. At 5 and 100 users the cached path was slower than the baseline. At 100 users, for example, mean latency rose from 15.779 to 26.364 ms.

The local MongoDB working set was already warm, and Redis introduced another service exchange and serialization path on the same machine. The evidence supports a database-load reduction at every level and a latency benefit only at the medium point. It does not support a general claim that caching always improves response time.

All rows have 100% request success. Cache hit/load counters include the complete run including ramp; HTTP timings use the steady measurement window. Cold-Redis population and JVM warm-up were recorded separately from the reported warm comparisons.

6.2 Resources and Redis failure

Table 8. Cache-study CPU (%) and RSS (MiB), aggregated over three repetitions [4].

Users / cache	App CPU	App RSS	DB CPU	DB RSS	Redis CPU	Redis RSS
5 / off	23.41	422.7	13.10	143.9	0.00	39.3
5 / on	24.99	433.9	0.49	144.2	1.65	39.2
25 / off	23.81	425.1	17.33	136.8	0.00	38.7
25 / on	27.61	437.6	0.50	135.4	1.77	38.7
100 / off	28.38	444.7	26.40	151.1	0.00	38.7
100 / on	25.66	440.7	0.48	151.3	1.44	38.8

CPU columns are means of normalized host-capacity percentages. RSS columns are the means of per-run observed peaks from summary.csv. They describe process observations, not per-request allocations. At 100 users, MongoDB mean CPU fell from 26.40% to 0.48% with cache enabled, while application CPU remained substantial.

Key-value lookup cost

Cache-on users	Mean cache lookup ms	JMeter CPU %	JMeter RSS MiB
5	0.406	6.86	303.3
25	3.666	8.09	295.9
100	22.596	8.45	293.5

The recorded mean cache lookup rises from 0.406 ms at 5 users to 22.596 ms at 100 users. This is compatible with contention or queueing along the cached path, but the observations do not isolate a single cause. Two backend JVMs were resident in the cache experiment, with traffic sent to one at a time.

Cache outage and recovery

Table 9. Separate 5-user Redis failure scenario; excluded from cache baselines.

State	Requests	Mean ms	p95 ms	Success/s	Error %	DB reads / req
Outage	15	4,049.07	4,082.00	1.2	0.00	1.0
Recovery	36,943	1.22	2.00	3,792.5	0.00	0.0

Product responses remained correct through MongoDB fallback. During outage the configured cache timeouts produced approximately four-second responses and very low throughput. After recovery and separate warm-up, responses returned to roughly 1.22 ms mean. A circuit breaker or shorter bounded timeout is a future improvement; the observed scenario validates cache fallback, not database failover.

6.3 Rate-limiter evaluation

Fixed pacing offered 10, 50 and 200 requests/s with 5, 25 and 100 users. All threads shared one client bucket, intentionally measuring enforcement of one quota. A five-second warm-up exhausted the initial burst allowance before the ten-second measured window.

Table 10. Admission, rejection and pacing under the token bucket [5].

Limiter	Users	Offered / s	Admit / s	Admit p95 ms	Reject %	Pacing p95 ms
Off	5	10	10.0	11.00	0.00	9.10
Off	25	50	50.0	18.00	0.00	16.00
Off	100	200	200.0	41.00	0.00	42.00
On	5	10	5.0	20.55	50.00	5.05
On	25	50	5.0	55.10	90.00	20.05
On	100	200	5.0	115.45	97.50	50.05

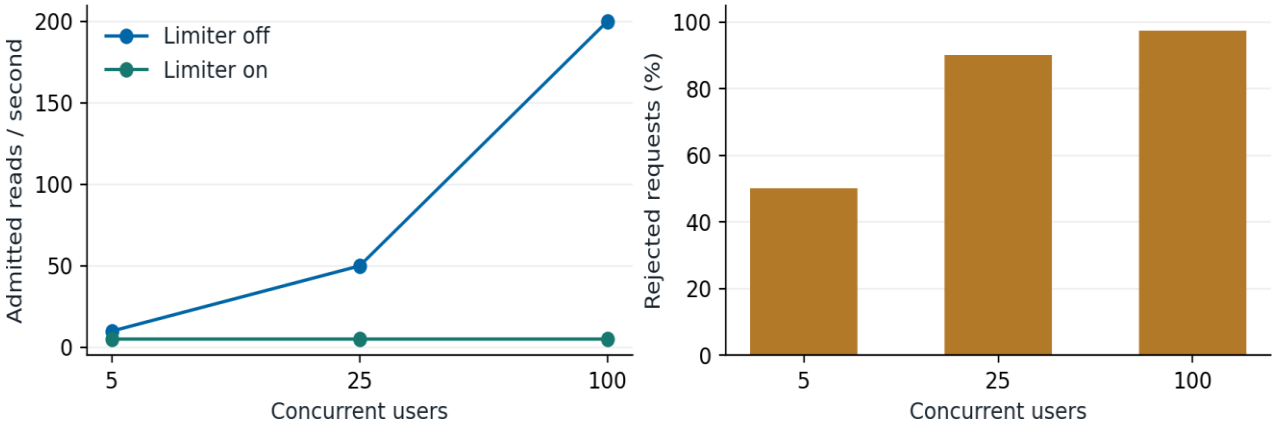


Figure 5. Steady admission remains at 5 reads/s while the rejection fraction grows.

Observed behavior

With the limiter disabled, actual and admitted rates matched each offered level. With it enabled, admission stayed at 5/s and rejection rose to 50%, 90% and 97.5%. The corresponding successful-admission fractions were 50%, 10% and 2.5%. All six runs reported zero unexpected errors, and a cooldown check admitted the same client again.

Pacing p95 lag stayed below 51 ms, providing evidence that the load generator reached its offered rates. HTTP 429 is an intentional rejection and is not counted as completed product work. Rejected requests still consume some interceptor and networking resources.

> **Trade-off:** the limiter enforces a stable per-client admission bound and makes overload visible. This experiment does not measure fairness across many clients or aggregate enforcement across multiple backend processes.

6.4 Logical-shard read evaluation

Both modes contained 100 products across five warehouses, giving 500 inventory records with 1,000 units each. Requests alternated a product/warehouse point read and a product-wide query. Cache and rate limiting were disabled.

Table 11. One-store versus three-logical-shard inventory reads [5].

Mode	Users	Point mean	Point p95	Wide mean	Wide p95	Reads / s
Single	5	14.93	24.00	13.21	21.00	330.0
Single	25	31.85	45.00	25.57	36.00	847.4
Single	100	103.57	126.00	80.03	100.00	1,067.6
3 shards	5	12.71	18.00	13.84	20.00	351.3
3 shards	25	39.83	53.00	46.83	63.00	564.6
3 shards	100	120.38	160.00	142.83	187.00	743.8

All latency columns are milliseconds. All rows report 100% successful reads and zero unexpected errors; each point uses one ten-second measurement window.

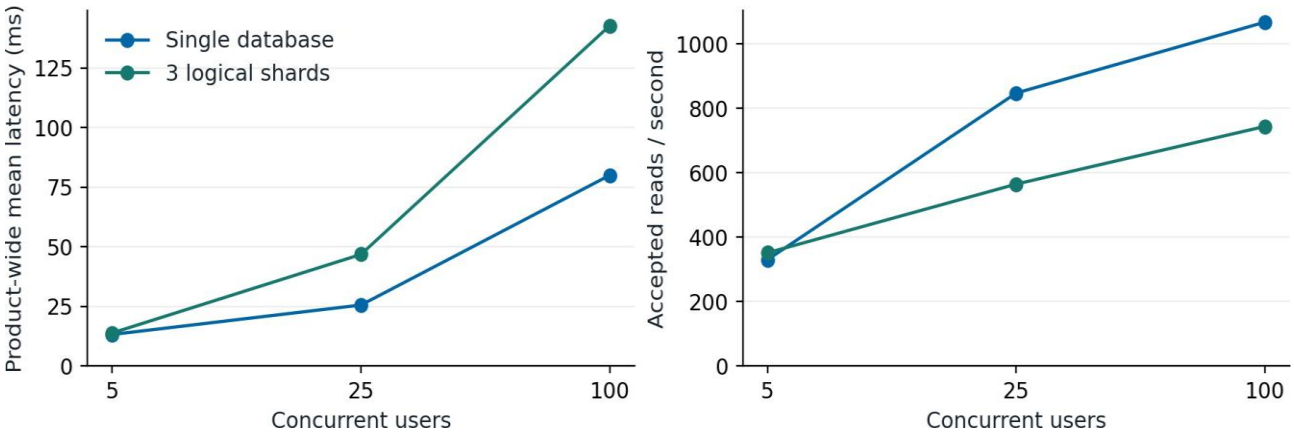


Figure 6. Product-wide latency and total read throughput on the shared MongoDB host.

At 100 users, point-read mean latency rose from 103.57 to 120.38 ms with sharding. Product-wide reads rose from 80.03 to 142.83 ms, approximately 78.5%, and accepted throughput fell from 1,067.6 to 743.8 reads/s. The wide query must consult all owners. Routing and repeated database work do not add physical CPU or storage capacity.

Load distribution and resource context

For the 100-user sharded run, persistence-call deltas were 7,608 / 7,310 / 8,138 across shards 1-3, approximately 33.0% / 31.7% / 35.3%. These counters include ramp and fan-out operations and must not be equated with steady HTTP request counts. Backend CPU was 15.54% with 453.56 MiB peak RSS; MongoDB used 28.35% CPU and 172.09 MiB peak RSS. Small low-load differences are insufficient for a general speedup claim.

6.5 Stock correctness and contention

Each burst submitted 100 one-unit orders. Distributed-stock cases used 100 distinct records with 1,000 units each. Hot-stock cases targeted one record containing ten units. No automatic retries are hidden in these first-attempt HTTP outcomes.

Table 12. Hot-stock burst outcomes; final reserved stock agrees with successful orders [5].

Mode	Users	Reserved orders	No stock	Retryable writes	Orders / s	Final R
Single	5	10	50	40	15.77	10
Single	25	5	0	95	9.31	5
Single	100	4	0	96	20.10	4
3 shards	5	10	50	40	15.24	10
3 shards	25	6	0	94	11.56	6
3 shards	100	2	0	98	8.06	2

At 5 users both modes reserved ten orders, rejected 50 for insufficient stock and returned 40 retryable write conflicts. At 100 users only four single-store and two sharded orders reserved; the other first attempts encountered write conflicts. Correct rejection prevented overselling but did not consume all available stock.

Distributed-stock throughput

Concurrent users	Single-store orders/s	Sharded orders/s	Successful orders per case
5	72.15	80.13	100 / 100
25	183.49	164.20	100 / 100
100	212.31	196.85	100 / 100

All six distributed cases completed 100 reservations. All twelve stock reconciliations passed: stored quantities, reserved orders and reservation audits agreed, with no oversold units, orphan orders/audits or duplicate/wrong stock owners. Correctness is supported by persisted-state checks, not just HTTP response codes.

Success, errors and resources

For each hot-stock row, the reserved-order count is also its success percentage because there are 100 attempts. Expected conflicts remain rejected work; unexpected error counts were zero. At 100 users, distributed single/sharded sales used 10.18% / 10.03% backend CPU with 457.96 / 454.85 MiB peak RSS. Very short hot bursts have too few resource samples for reliable CPU conclusions.

> **Safety and progress differ:** stock bounds held. Bounded whole-request retries with stable idempotency keys need a separate evaluation of eventual fulfillment.

6.6 Hash distribution and migration

Table 13. Distribution of 100,000 deterministic keys across three shards [5].

Virtual nodes	Records: shard 1 / 2 / 3	CV	Ring remap %	Modulo remap %
1	64,425 / 29,490 / 6,085	0.719	39.20	74.96
32	34,499 / 27,715 / 37,786	0.126	26.77	74.96
128	29,647 / 32,573 / 37,780	0.101	25.90	74.96

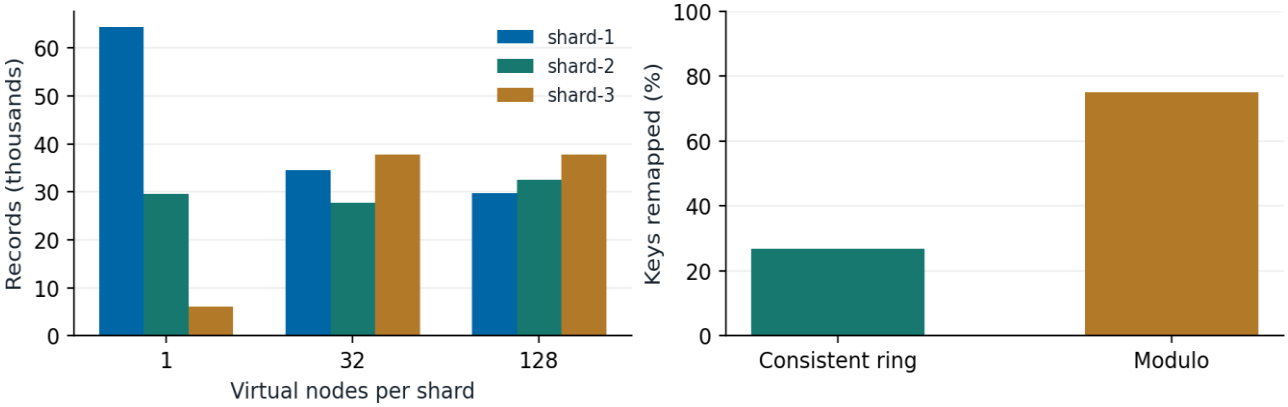


Figure 7. Record balance and three-to-four-shard key movement; CV = standard deviation / mean.

At the configured 32 virtual nodes, adding a fourth shard moved 26.775% of keys versus 74.959% for modulo placement. Increasing virtual nodes improved the sample's record balance, though 128 nodes were still not perfectly balanced. An 80%-hot-key simulation sent 87.51% of traffic to one owner at 32 virtual nodes, showing that balanced record counts cannot eliminate a hot key.

Live three-to-four-to-three membership change

Table 14. Graceful membership change during read traffic [5].

Change	Scanned	Moved	Remapped %	Server time ms
Add	803	213	26.53	2934
Remove	803	213	26.53	2774

All 803 full documents were preserved, including quantities, reservations, IDs, versions and timestamps; duplicate copies after completion were zero. The 25-user measurement recorded 7,162 accepted reads and 50 explicit snapshot/routing conflicts out of 7,212 attempts: 99.31% success. Maximum observed HTTP latency was 3,026 ms.

The migration lock explains the pauses. Source data stayed reachable during removal, so this experiment demonstrates controlled rebalancing and state preservation rather than failover after losing a machine. Orders and audits matched their pre-change snapshot.

6.7 Snowflake and Trie benchmarks

Table 15. Direct Java ID generation: 250,000 IDs per point, one repeat [5].

Generator	Threads	IDs / second	Collisions	CPU ms	ID chars
Uuid	1	1,403,997	0	578.12	42
Snowflake	1	4,035,454	0	234.38	24
Uuid	8	1,108,192	0	656.25	42
Snowflake	8	4,054,611	0	187.50	24
Uuid	32	1,059,173	0	671.88	42
Snowflake	32	3,401,444	0	265.62	24

Snowflake generated 3.40-4.05 million IDs/s versus 1.06-1.40 million for full UUIDs. All 1.5 million normal benchmark IDs passed collision checks. A fixed-clock misconfiguration test intentionally produced 1,000 collisions with duplicated node identities; distinct node IDs produced none. ID lengths include the benchmark prefix. The whole benchmark JVM peaked at 325.76 MiB RSS.

Table 16. Actual ProductTrie versus pre-tokenized linear search, 500 queries per point [5].

Products	Prefix	Linear mean ms	Trie mean ms	Trie p95 ms	Set accuracy
100	selective	0.0095	0.0015	0.0015	100%
100	broad	0.0015	0.0091	0.0141	100%
1,000	selective	0.0381	0.0010	0.0010	100%
1,000	broad	0.0021	0.0730	0.1433	100%
10,000	selective	0.3798	0.0011	0.0013	100%
10,000	broad	0.0003	1.4315	2.9160	100%

For 10,000 products, selective lookup averaged 0.0011 ms with the Trie versus 0.3798 ms with linear scanning. Broad prefixes reversed the result: 1.4315 ms versus 0.0003 ms. The linear top-eight baseline stops early, whereas the Trie collects all matching candidates before slicing the result.

Accuracy, build cost and scope

Full candidate-set comparisons and top-eight count/validity checks passed, including missing-prefix cases. Returned order has no equivalence contract. At 10,000 products, Trie construction took 1,348.00 ms versus 21.39 ms for the linear index. Approximate heap growth was 12.73 versus 8.74 MiB, including temporary allocations and garbage collection effects. The Node process peaked at 156.56 MiB RSS.

These algorithm timings exclude HTTP and browser rendering. Sub-microsecond observations are sensitive to timer, warm-up and loop effects; a single repeat is an initial comparison rather than a stable application response-time guarantee.

6.8 Verification and evidence coverage

Backend correctness evidence

The latest saved correctness run reports **85 tests passed, 0 failures, 0 errors and 0 skipped** across 19 test classes [6]. It includes 20 real replica-set consistency integration tests. Unit and integration coverage is summarized below.

Table 17. Selected executed correctness areas and their assertions.

Area	What the saved tests check
Stock and orders	Concurrent reservations, non-negative stock, order/audit reconciliation and multi-item failure rollback.
Idempotency / receipts	Same-key behavior, replay-safe receipt credit, duplicate-line aggregation and receipt-save rollback.
Transfers / lifecycle	Valid dispatch-before-completion, failure rollback and concurrent terminal actions that conserve stock.
Sharding	Deterministic routing, raw-document preservation, add/remove/restart and interrupted migration recovery.
Cache / alerts / IDs	Authoritative availability, cache adapter behavior, alert uniqueness and Snowflake checks.

Supplier-crawler observations

The existing supplier-component document reports a combined sweep of four pages and 24 valid items in 271 ms: 88.6 items/s, zero rejected items and 100% reported extraction accuracy [7]. It also reports seven passing crawler tests. These are document-reported observations; the referenced report does not provide the same raw-run manifest, repetitions or resource series as the October 24 benchmark bundles.

Unmeasured or incompletely verified points

The shortener exposes redirect timing and collision counters, but no saved redirect workload series is included. Source inspection also finds zero-offset TTL index annotations on both createdAt and expiresAt. The createdAt index can make a mapping eligible for early deletion; the nominal 90-day lifetime must not be treated as verified until index configuration and expiry behavior are corrected and tested.

Real multi-member primary step-down, unknown-commit network failures, multi-backend membership coordination, catalogue coverage in the browser, and supplier-import atomicity/replay remain outside the measured evidence. Correctness-suite timings are not throughput measurements. Some database tests retry aborted transactions; that explains why their outcomes differ from first-attempt HTTP contention results.

Evidence conclusion: the report covers all three workload levels, multiple baselines, resource observations, stock reconciliation, shard changes and a cache outage. Missing component metrics and untested failure modes are explicit rather than inferred.

7 Scalability, reliability and trade-offs

Table 18. Observed bottlenecks and justified next steps.

Observation	Design trade-off	Next engineering step
Logical shards share resources	Partition routing is demonstrated, but one MongoDB deployment supplies capacity.	Evaluate native physical sharding or replicated placement with explicit transaction semantics.
Product-wide reads fan out	Point locality is gained at the cost of multi-owner queries.	Batch and parallelize bounded reads, paginate results and evaluate access-pattern-specific indexes.
Hot stock causes many conflicts	Conditional writes and transactions favor stock safety over first-attempt admission.	Measure bounded backoff/retry and fair admission using stable request identities.
Migration pauses inventory	A local lock simplifies preservation and recovery.	Stream/batch copies and design versioned online routing with coordination and fencing.
Cache outage is slow	Database fallback preserves responses while cache timeouts dominate latency.	Add a measured circuit-breaker policy and bound fallback latency.
Broad Trie prefixes are expensive	Subtree collection precedes top-eight truncation.	Maintain bounded suggestions or stop traversal early under a defined ranking contract.

7.1 Reliability boundaries

The local router lock and token buckets are process-scoped. Multiple application instances would need coordinated membership, unique Snowflake node identities and a deliberate rate-limit policy. A one-member replica set does not tolerate loss of its host. Graceful shard removal requires accessible source data; replication and failover must be designed separately.

7.2 Validity of the evaluation

All services and JMeter share one Windows host with background applications. The component suite uses short runs and one repetition; cache comparisons have three repetitions but still share resources and a warm database working set. Algorithm tests exclude network and UI costs. These conditions explain why larger concurrency does not yield a smooth or universally beneficial performance trend.

7.3 Remaining functional design work

Orders and movement data need a partitioning strategy to complete the assignment's transaction-data requirement. Reservation expiry is deferred. Large receipt histories would benefit from a bounded receipt ledger, while supplier imports need a durable batch identity and clearer atomicity. Short-link lifetime behavior needs correction and dedicated measurement before a retention guarantee is made.

8 End-to-end demonstration

The following sequence demonstrates the integrated prototype and connects each visible action to a correctness or system-design observation. Example quantities are illustrative and can be entered through the UI after the environment is ready.

Table 19. Suggested demonstration sequence (approximately 10-15 minutes).

Step	Action	What to show
1	Register one product and two warehouses.	Generated IDs, unique SKU validation and initially empty warehouse stock.
2	Create a 40-unit purchase order and receive it.	CREATED to RECEIVED; 40 physical/available units and a purchase movement.
3	Search the product by SKU/name prefix.	Trie suggestions, product selection and authoritative warehouse quantities.
4	Reserve a 4-unit sales order, then complete it.	P=40, R=4, A=36 while reserved; P=36, R=0 after completion.
5	Transfer 5 units to the second warehouse.	Source falls to 31 while 5 are in transit; completion credits destination to 5.
6	Take availability to the reorder threshold.	One active low-stock alert; replenishment above threshold resolves it.
7	Inspect routes and activate/deactivate shard 4.	Record/request distribution, movement percentage and preserved stock after both changes.
8	Run supplier discovery and review/import a feed.	Different portal formats, crawl telemetry and no physical-stock credit before receipt.
9	Demonstrate short-link creation and resolution.	302 response, target, click counter and timing header; validate current expiry behavior.
10	Open the saved evaluation evidence.	Three workload levels, expected rejections, stock checks, migration and Redis recovery.

Useful API and script entry points

Core: /api/products, /api/warehouses, /api/inventory, /api/purchase-orders, /api/sales-orders, /api/transfers and /api/alerts/low-stock.
Components: /api/shards, /api/shards/route, /api/shorten, /r/{code} and /api/suppliers/import.
Scripts: scripts/demo-sharding.ps1; scripts/e2e_integration_demo.js; scripts/test-consistency.ps1; scripts/test-components.ps1.

Use transaction-capable MongoDB for the backend and the supplier portal on port 8085 for crawler demonstrations. The sharding demo creates sample records and changes membership, so run it against the selected demonstration dataset. Saved benchmark results can be presented directly alongside live workflow observations.

9 Conclusion and references

AetherStock integrates the principal inventory workflows with eight system-design components and an explicit stock-consistency model. Shared MongoDB transactions and guarded mutations protect business state, while consistent hashing supports controlled logical partitioning. The saved tests and reconciliations demonstrate stock safety and document preservation in the executed scenarios.

The evaluation's main lesson is that component benefits depend on workload and deployment. Redis removes product database reads but does not always improve latency; logical sharding adds overhead without new physical resources; hot-stock transactions preserve safety while limiting immediate progress. The prototype therefore meets the goal of an evidence-based micro-project demonstration, with transaction-data partitioning, physical scale-out and remaining verification gaps clearly identified as future work.

References and evidence index

[1] Sri Sivasubramaniya Nadar College of Engineering. *Mini Project Problem Statements*, UCS3513 System Design Laboratory, Topic 4, pp. 8-9, 2026-27. Repository: Mini Project Problem Statements.pdf.

[2] AetherStock implementation and consistency design. backend/src/main/java/com/example/inventory/; docs/consistency.md; backend/src/main/resources/application.yml.

[3] AetherStock logical-sharding design and recovery protocol. docs/sharding.md; backend/src/main/java/com/example/inventory/sharding/.

[4] Product-cache benchmark bundle, 24 October 2026. evaluation/load-results/20261024-130716-cache/: README.md, manifest.json, summary.csv, redis-failure-check.json and per-run raw/resource observations.

[5] Component benchmark bundle, 24 October 2026. evaluation/results/20261024-145210-components/: README.md, FINDINGS.md, manifest.json, http-results.json, migration-check.json, hashing.json, ids.json, autocomplete.json and per-run stock checks. Runner: evaluation/QUICK-TESTS.md.

[6] Correctness evidence, 24 October 2026. evaluation/results/20261024-125728-079-consistency/README.md: 85 passing backend tests and per-class timings.

[7] Supplier integration and reported crawler observations. docs/member-5-supplier-crawler-and-integration.md; crawler/crawler_engine.js; crawler/test_crawler.js. Autocomplete implementation: frontend/src/services/trie.js.

[8] MongoDB, *Transactions*. https://www.mongodb.com/docs/manual/core/transactions/. Spring Data MongoDB transaction support: official session and transaction reference.

[9] Apache JMeter, *User Manual*. https://jmeter.apache.org/usermanual/. Institutional logo asset: SSN official website, https://www.ssn.edu.in/. Logo source retrieved 25 October 2026.